# Try & Except Basics

**Error handling** is the practice; **exception handling** is Python's try/except tool.

## 1) An uncaught error stops the program

**Example :**

```
print(10 / 0)
```

**Output :**

```
ZeroDivisionError: division by zero
```

## 2) Handle it with try / except

**Example :**

```
try:
    a = 10
    b = 0
    result = a / b
    print("Result is", result)
except ZeroDivisionError:
    print("Error: zero division not allowed")
print("Program Completed")
```

**Output :**

```
Error: zero division not allowed
Program Completed
```

*the code after the try block still runs, so the program is not terminated*

## 3) Handle invalid input: ValueError and else

**Run 1 : valid**

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Enter the valid age")
else:
    print(f"The Age is: {age}")
```

**Output :**

```
Enter your age: 43
The Age is: 43
```

**Run 2 : text**

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Enter the valid age")
else:
    print(f"The Age is: {age}")
```

**Output :**

```
Enter your age: abc
Enter the valid age
```

*else runs only when no error happened, except runs only when the matching error did*

## Quick check

`int("abc")` raises **ValueError**; `10 / 0` raises **ZeroDivisionError**.

# Exceptions, Else & Finally

Different problems raise different exceptions. Catch the one you expect.

## 1) Common exceptions

| Exception | Raised when | Example | Message |
|---|---|---|---|
| ZeroDivisionError | dividing by zero | 10 / 0 | division by zero |
| ValueError | text is not a number | int("abc") | invalid literal for int() w... |
| FileNotFoundError | file does not exist | open("nofile.txt") | [Errno 2] No such file or d... |
| TypeError | wrong type combined | "Age: " + 19 | can only concatenate str (n... |
| IndexError | index out of range | [1, 2][5] | list index out of range |
| KeyError | missing dictionary key | {"a": 1}["b"] | 'b' |

## 2) FileNotFoundError and the x mode

**Example :**

```
try:
    with open("missing.txt", "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("File not Found. Enter the right location")
```

**Output :**

```
File not Found. Enter the right location
```

**Example :**

```
for attempt in range(2):
    try:
        with open("Practice_New_File.txt", "x", encoding="utf-8") as f:
            f.write("Python practice file")
        print("New file created!")
    except FileExistsError:
        print("File exists. It was not overwritten.")
```

**Output :**

```
New file created!
File exists. It was not overwritten.
```

*"x" mode creates a file only if it does not exist, otherwise Python raises FileExistsError*

## 3) finally always runs

**Example :**

```
try:
    number = int(input("Enter an integer: "))
except ValueError:
    print("That was not a valid integer.")
else:
    print("You entered:", number)
finally:
    print("Program finished.")
```

**Output :**

```
Enter an integer: x7
That was not a valid integer.
Program finished.
```

# Raise & Validators

Use `raise ValueError("message")` to reject a value that Python itself would accept.

## 1) Marks validator

**Run 1 : 105**

```
try:
    marks = float(input("Enter marks between 0 and 100: "))
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100.")
except ValueError as error:
    print("Invalid marks:", error)
else:
    print("Valid marks:", marks)
    print("Pass" if marks >= 35 else "Fail")
```

**Output :**

```
Enter marks between 0 and 100: 105
Invalid marks: Marks must be between 0 and 100.
```

**Run 2 : 72**

```
try:
    marks = float(input("Enter marks between 0 and 100: "))
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100.")
except ValueError as error:
    print("Invalid marks:", error)
else:
    print("Valid marks:", marks)
    print("Pass" if marks >= 35 else "Fail")
```

**Output :**

```
Enter marks between 0 and 100: 72
Valid marks: 72.0
Pass
```

*except … as error gives you the message that was raised, or Python's own*

## 2) Bank withdrawal

**Example :**

```
balance = 10000
try:
    amount = float(input("Enter withdrawal amount: "))
    if amount <= 0:
        raise ValueError("Withdrawal amount must be greater than zero.")
    elif amount > balance:
        raise ValueError("Insufficient balance.")
except ValueError as error:
    print("Transaction failed:", error)
else:
    balance -= amount
    print(f"Withdrawal successful! Remaining: Rs.{balance:.2f}")
```

**Output :**

```
Enter withdrawal amount: 20000
Transaction failed: Insufficient balance.
```

### ☆ WATCH OUT

- Test a range with a chained comparison: `18 <= age <= 35`, or `age >= 18 and age <= 35`.
- Do not write `age >= 18 & age <= 35`: `&` is a bitwise operator and gives wrong results.

# Practice & Revision

Write each handler yourself first, then compare. Each solution is run for real.

## Task 1 Division calculator.

**Solution :**

```
try:
    first = float(input("First number: "))
    second = float(input("Second number: "))
    result = first / second
except ZeroDivisionError:
    print("You cannot divide by zero.")
except ValueError:
    print("Please enter valid numbers.")
else:
    print("Result:", result)
```

```
First number: 10
Second number: 0
You cannot divide by zero.
```

## Task 2 Valid age from 0 to 120.

**Solution :**

```
try:
    age = int(input("Enter your age: "))
    if age < 0 or age > 120:
        print("Invalid age.")
    else:
        print("Your age is:", age)
except ValueError:
    print("Please enter a whole number.")
```

```
Enter your age: abc
Please enter a whole number.
```

## Task 3 Convert a list safely.

**Solution :**

```
values = ["10", "25", "hello", "40"]
for value in values:
    try:
        print(int(value))
    except ValueError:
        print("Cannot convert:", value)
```

```
10
25
...
```

## Task 4 Read a file safely.

**Solution :**

```
filename = input("Filename: ")
try:
    with open(filename) as file:
        print(file.read())
except FileNotFoundError:
    print("File not found.")
```

```
Filename: nofile.txt
File not found.
```

## Quick revision

| Keyword | Role |
|---|---|
| `try` | the risky code |
| `except` | runs only if that error happens |
| `else` | runs only if no error happened |
| `finally` | always runs, error or not |
| `raise` | create an error yourself to reject a bad value |